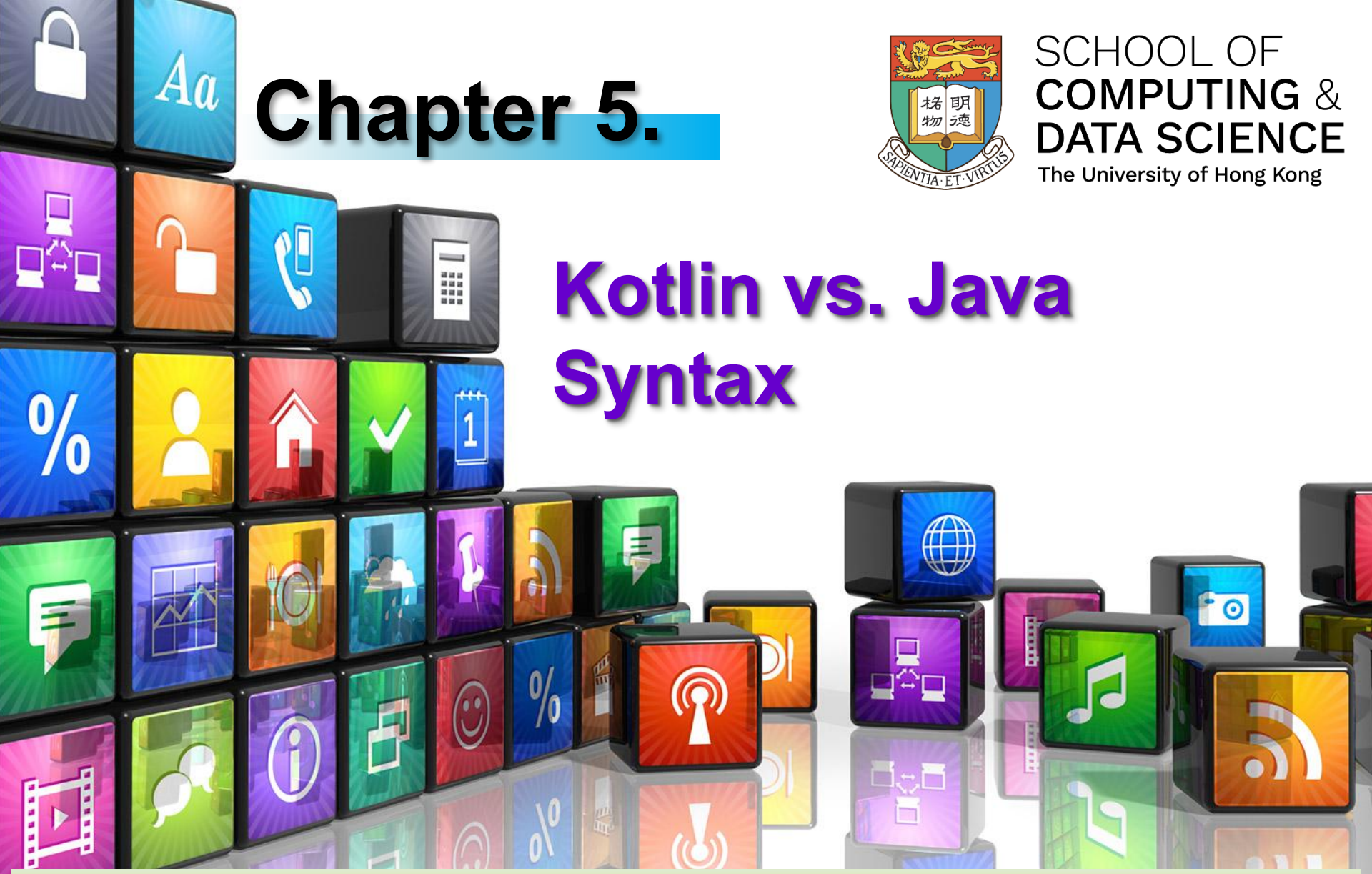
Chapter 5.

SCHOOL OF COMPUTING & DATA SCIENCE
The University of Hong Kong

Kotlin vs. Java Syntax



2026-2027

COMP3330 Interactive Mobile Application Design and Programming

Dr. T.W. Chim (E-mail: twchim.cs@hku.hk)

Agenda

- Data Types
- Functions & Function Calls
- Variables (Immutable vs. Mutable)
- Kotlin Arrays
- Null Safety in Kotlin
- String Interpolation
- Kotlin Classes

Data Types

- Recall that Java supports 8 primitive types, and each primitive type has the corresponding wrapper class.

Primitive Type	Wrapper Class	Bit Depth	Value Range
boolean	Boolean	JVM-specific	true or false
char	Character	16 bits	0 to 65535
byte	Byte	8 bits	-128 to 127
short	Short	16 bits	-32768 to 32767
int	Integer	32 bits	-2147483648 to 2147483647
long	Long	64 bits	-huge to huge
float	Float	32 bits	varies
double	Double	64 bits	varies

Data Types

- Kotlin doesn't have primitive types. It uses classes as an object wrapper for primitives.

Type	Bit Depth	Value Range
Boolean	JVM-specific	true or false
Char	16 bits	0 to 65535
Byte	8 bits	-128 to 127
Short	16 bits	-32768 to 32767
Int	32 bits	-2147483648 to 2147483647
Long	64 bits	-huge to huge
Float	32 bits	varies
Double	64 bits	varies
String	variable	varies

Functions

- Java

```
public int sum(int a, int b) {
        return a + b;
}
```

- Kotlin

```
public fun sum(a: Int, b: Int): Int {
        return a + b
}
```

Key differences:

- Keyword “fun”
- Parameter definition (colon notation)
- Return type indication (colon notation)
- Absence of semi-colon

Functions

- Interestingly, the following Kotlin function definitions are equivalent.

```
public fun sum(a: Int, b: Int): Int {
        return a + b
}

public fun sum(a: Int, b: Int): Int = a + b

public fun sum(a: Int, b: Int) = a + b
```

Function Calls

- Java

```
public void calculate(int num1, int num2, int num3) {
        …
}
→ calculate(1, 3, 5);
```

- Kotlin

```
fun calculate(num1: Int, num2: Int, num3: Int) {
        …
}
→ calculate(1, 3, 5)
```

Note that this is a void function with no return value.

Function Calls

- In Kotlin, we can provide a default value for a function parameter.
- In the following, num2 will hold the provided value 3 (because value is provided) while num3 will hold the default value 4 (because value is not provided).

```
fun calculate(num1: Int, num2: Int = 2, num3: Int = 4) {
        …
}
→ calculate(1, 3)   // 3rd parameter missing!
```

Challenge: How to provide 1st and 3rd parameters while missing 2nd parameter?

Function Calls

- The following Kotlin function calls are possible. Parameter names can be used in function calls.

```
fun calculate(num1: Int, num2: Int = 2, num3: Int = 4) {
        …
}
```

→ calculate(1, 3) // num3 will be assigned the default value 4

→ calculate(1, num2 = 3) // num3 will be assigned the default value 4

→ calculate(1, num3 = 5) // num2 will be assigned the default value 2

Variables

- Java

```
int a = 1;
String b = “tim”;
```

- Kotlin (**Immutable** Version 1)

```
val a: Int = 1
val b: String = “tim”
```

- Kotlin (**Mutable** Version 1)

```
var a: Int = 1
var b: String = “tim”
```

- Kotlin (**Immutable** Version 2)

```
val a = 1   // Inferred type is Int
val b = “tim”   // Inferred type is String
```

- Kotlin (**Mutable** Version 2)

```
var a = 1   // Inferred type is Int
var b = “tim”   // Inferred type is String
```

Value re-assignment to a and b will cause error!

Kotlin Arrays

Recall that in Java, we can create an array of integers like:
int[] num = {1, 2, 3, 4};
OR
int[] num;
num = new int[4];
num[0] = 1; num[1] = 2; num[2] = 3; num[3] = 4;

- Using the **arrayOf()** function

```
val num = arrayOf(1, 2, 3, 4)   //implicit type declaration
val num = arrayOf<Int>(1, 2, 3, 4)   //explicit type declaration
```

- Traversing an array and accessing array elements

```
for (i in 0..num.size-1) {
        print(" “ + num[i])
}
```

Kotlin Arrays

- Using the Array constructor
  - Since Array is a class in Kotlin, we can also use the Array constructor to create an array.
  - The constructor takes two parameters:
    - The **size of the array**, and
    - A function which accepts the **index of a given element** and returns the **initial value of that element**.

```
val num = Array(3, {i-> i*1})
// equivalent to val num = arrayOf(0, 1, 2) / val num = arrayOf<Int>(0, 1, 2)
```

Null Safety in Kotlin

- In Kotlin, the type system distinguishes between references that can hold null (nullable references) and those that cannot (non-null references). A regular variable of type String cannot hold null by default and the following will cause compilation error.

```
var str: String = “hku”
str = null
```

- To allow nulls, we can declare a variable as nullable by writing **?** after the type.

```
var str: String? = “hku”
str = null
```

Null Safety in Kotlin

- If we call a method or access a property on a non-null reference, it's guaranteed not to cause a Null Pointer Exception (NPE). The following is safe.

```
var S1: String = “hku”
val l = S1.length
```

- If we call a method or access a property on a nullable reference, it would not be safe. The compiler reports an error.

```
var S2: String? = “hku”
val l = S2.length   // error: variable ‘S2’ can be null
```

Null Safety in Kotlin

- For nullable reference, we need to explicitly check whether the reference is null and handle the two options separately.

```
var S2: String? = "hku"
val l = if (S2 != null) S2.length else null
```

In Kotlin, it is possible to assign an if condition to a variable!

- To simplify and to get rid of the if condition, we can make use of **safe calls**. l is assigned null if S2 is null. So the type of l is Int?.

```
var S2: String? = "hku"
val l = S2?.length
```

Null Safety in Kotlin

- Another example of safe calls:

```
val a = "Computer"
val b: String? = null
println(b?.length)
println(a?.length)   // Unnecessary safe call as a is non-null
```

Output:

```
null
8
```

Note: The type of expressions a?.length and b?length is Int?.

Null Safety in Kotlin

- Safe calls can be used in chains. For example, Tim is an employee who may be assigned to a department (or not). That department may in turn have another employee as a department head (or not). To obtain the name of Tim's department head (if any), we write the following:

```
tim?.department?.head?.name
```

Such a chain returns null if any of the properties in it is null.

Null Safety in Kotlin vs. Java

- Java

```
public ZipCode getZipCode(User user) {
        if (user != null) {
                if (user.address != null) {
                        return user.address.zipCode;
                }
        }
        return null;
}
```

- Kotlin

```
fun getZipCode(user: User?): ZipCode? {
        return user?.address?.zipCode
}
```

or

```
fun getZipCode(user: User?) = user?.address?.zipCode
```

Java: Several lines of code
Kotlin: Single line of code

String Interpolation

- The following three functions are equivalent. We want to compose a string with 3 variables inside. **$** can be used to interpolate variables within a string.

```
override fun toString(): String {
        return “Song{id=“ + id + “, title=‘” + title + “’, author=‘” + author + “’}”
}

override fun toString(): String {
        return “Song{id=$id, title=‘$title’, author=‘$author’}”
}

override fun toString() = “Song{id=$id, title=‘$title’, author=‘$author’}”
```

Kotlin Classes

Like Java, the compiler will generate an additional constructor without parameters on the JVM. For this purpose, it'll instantiate an object with default values.

- A Kotlin basic class declaration, with two properties and a primary constructor can be written as follows:

```
class Person constructor(val name: String, val age: Int? = null)
```

or

```
class Person(val name: String, val age: Int? = null)
// name and age are read-only here
```

- Like Java, we can create a new instance of a class. However, there is no "new" keyword in Kotlin. Instantiation is like constructor invocation.

```
val person1 = Person("Sam", 20)
val person2 = Person("Priscilla")   // age is assigned null value
```

Kotlin Classes

- To do the same in Java, we need much more code!

```
class Person {
        final String name;
        final Integer age;
        public Person(String name) {
                this.name = name;
                this.age = null;
        }
        public Person(String name, Integer age) {
                this.name = name;
                this.age = age;
        }
}
```

If you can remember Kotlin syntax well, a Kotlin program can be much shorter than its Java counterpart.

Kotlin Classes

- We cannot put any code in the primary constructor.
- If we want to execute some initialization code, we have to put them in an **initializer block**, which is prefixed with the “init” keyword.
- The initializer block is called after the primary constructor. We can also access class fields there.

```
class Person(val name: String, val age: Int? = null) {
        init {
                println("Hello, I'm $name")
        }
}
```

val person1 = Person(“Sam”, 20) will produce the following on screen.

Hello, I’m Sam

Kotlin Classes

- In a Kotlin class, we can declare one or more secondary constructors. Secondary constructors are prefixed with the **constructor** keyword.

```
class Car {
        val id: String
        val type: String
        constructor(id: String, type: String) {
                this.id = id
                this.type = type
        }
}
```

We can use a trailing comma when declaring class properties:

```
val id: String,
val type: String,
```

Sample usage:

```
fun main(args: Array<String>) {
    val car1 = Car("1", “BYD")
    val car2 = Car("2", “Isuzu")
}
```

Kotlin Classes

- In a Kotlin class, a secondary constructor can invoke the primary constructor using **this** keyword.

```
class Car(val id: String, val type: String) {
        constructor(id: String): this(id, "unknown")
}
```

Kotlin Classes

- We can use a primary constructor of the superclass.

```
class Employee(name: String, val salary: Int): Person(name)
```

By doing this, we pass a name to the primary constructor of Person class. Additionally, we add a new field called salary in the Employee class.

Note: All classes in Kotlin are final by default. We need to add the **open** keyword so that we can inherit from our Person class.

Kotlin Class Inheritance (Example)

```
open class Polygon {
        open fun draw() {
                // some default polygon drawing method
        }
}

abstract class WildShape : Polygon() {
        // Classes that inherit WildShape need to provide
        // their own draw method instead of using the
        // default on Polygon
        abstract override fun draw()
}
```

Java version:

```
class Polygon {
   void draw() {
      …
   }
}

abstract class WildShape extends Polygon
{
   @override
   abstract void draw() {
      …
   }
}
```

Kotlin Class Inheritance (Example)

```
abstract class Polygon {
        abstract fun draw()
}

class Rectangle : Polygon() {
        override fun draw() {
                // draw the rectangle
        }
}
```

Java version:

```
abstract class Polygon {
   abstract void draw() {
      …
   }
}

class Rectangle extends Polygon {
   @override
   abstract void draw() {
      …
   }
}
```

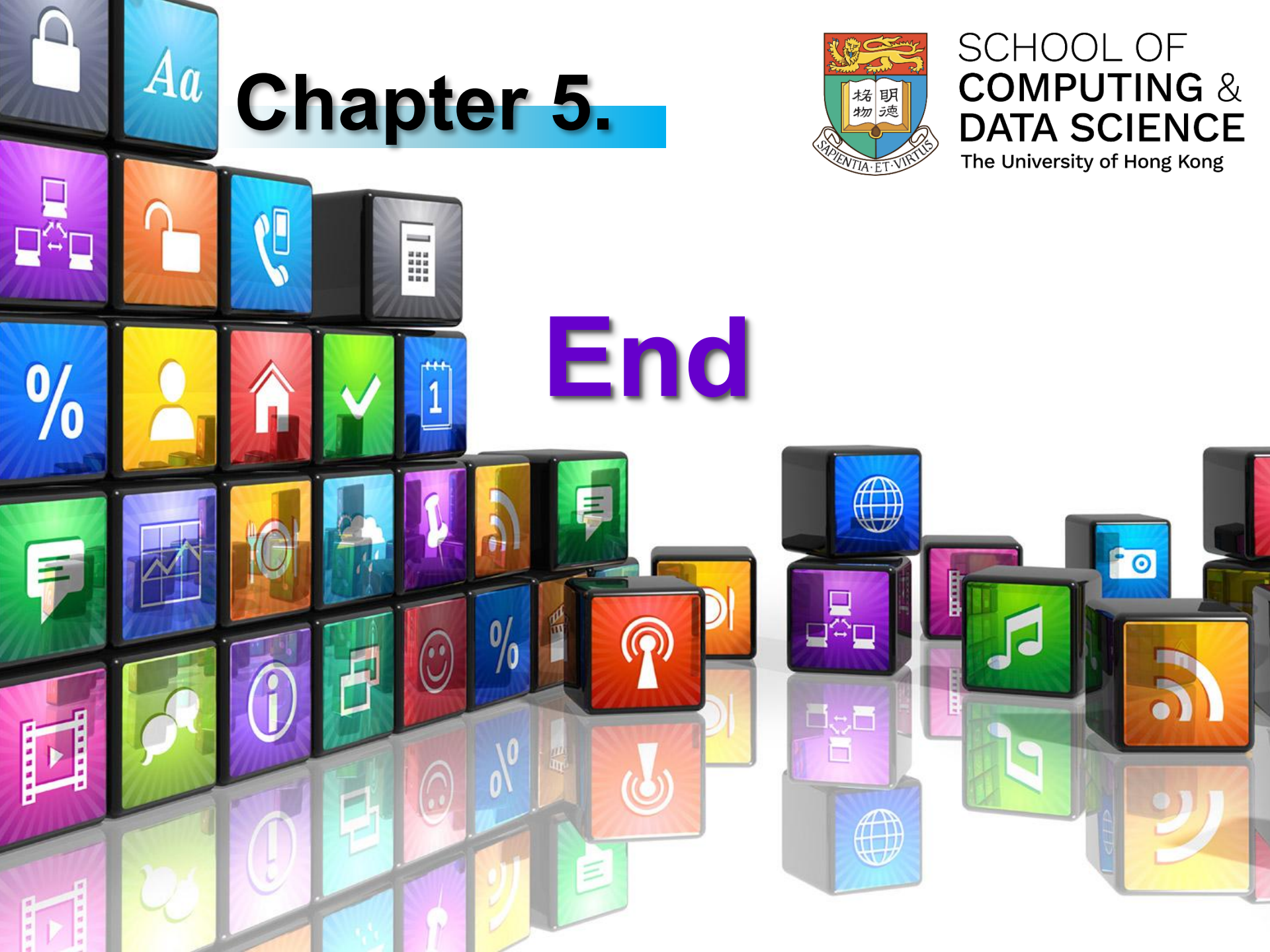
We are with you!



If you encounter any problems in understanding the materials in the lectures, **please feel free to contact me or my TAs**.
We are always with you!

We wish you enjoy learning Kotlin in this class.

Chapter 5.



SCHOOL OF COMPUTING & DATA SCIENCE
The University of Hong Kong

End